

八. STM32 单片机阶段

1. 学习定位总结

(加粗的面试常问!!! 包括英文全称要会读出来)

1.1 STM32 外设通信总线的复习小结:

GPIO: 了解基础知识、工作模式、电气特性与配置和应用场景。

USART/UART/RS232/RS485:

- **通信方式:** USART 支持同步和异步通信, UART 仅支持异步通信, USART 和 UART 都是串口通信协议, 而 RS232 和 RS485 则是具体的电气接口标准。
- **电平标准与兼容性:** RS232 电平值较高, 与 TTL 不兼容; RS485 电平值较低, 与 TTL 兼容。
- **通信距离与速度:** RS232 传输距离短, 速度相对较低; RS485 传输距离远, 速度快。
- **应用场景:** , USART/UART 通常与 RS232/RS485 等接口标准结合使用, UART 常用于嵌入式系统中的主机与辅助设备之间的通信; RS232 适用于点对点通信; RS485 适用于多设备通信和长距离传输。

IIC: 了解工作原理、通信流程和**数据帧**, 掌握配置和使用方法, 包括起始信号、停止信号、应答信号等。

SPI: 通信协议的特点、工作原理和**数据帧**, 掌握配置和使用方法, 包括主从模式、工作模式、时钟极性、时钟相位等参数, **还有和 IIC 的区别是?**

CAN: 总线的特点和应用场景, 了解 CAN 通信协议的**基本概念和帧结构**, 掌握配置和使用方法。

1.2 STM32 外设模块的复习小结:

DMA: 理解工作原理、配置方法和应用场景。

ADC/DAC: 理解工作原理、熟悉特性、配置方法和应用场景。

2.1 STM32 的 Hal 库和标准库有什么区别/优缺点? (如果问)

在面试中, 当被问及 STM32 的 HAL 库和标准库的区别及优缺点时, 可以参考以下回答结构:
“STM32 的 HAL 库和标准库在接口抽象层次、功能范围、资源占用等方面存在差异。

HAL 库是一个高级抽象层库, 提供了易于使用的函数接口和**丰富的功能支持**, 适用于需要快

速开发和原型验证的项目，以及对硬件抽象层有较高需求的开发者。

然而，**HAL 库**的代码体积相对较大，执行效率可能稍低。

相比之下，**标准库**是一个低级接口库，提供了与硬件直接交互的**底层接口函数**，具有资源占用小、性能优越等优点。

但标准库的**维护较为困难**，且**功能相对单一**，需要开发者自己编写更多的代码来实现高级功能。

因此，在选择使用哪种库时，需要**根据具体的应用需求**和开发者的技术水平来决定。”

这样的回答既全面又具体，能够清晰地阐述 HAL 库和标准库的**区别及优缺点**，同时展现出你对 STM32 开发库的理解和应用能力。

2. 定时器总结

2.1 定时器概述

STM32 系列微控制器中的定时器是非常重要的外设，它们具有多种功能，可以满足不同的应用需求。定时器主要分为**高级定时器**、**通用定时器**和**基本定时器**三类。其中，**通用定时器**在大多数应用中已经足够使用，因此是面试复习的重点。

2.2 定时器的种类与功能

基本定时器（如 TIM6、TIM7）：主要功能就是定时，由各自的可编程预分频器驱动。它们通常用于简单的**定时任务**，如**延时**等。

通用定时器（如 TIM2~TIM5）：除了基本的定时功能外，还具有**输入捕获**、**输出比较**和产生 PWM 波形等功能。这些功能使得通用定时器在**测量信号频率**、**脉冲宽度**以及**生成控制信号**等方面具有广泛的应用。

高级定时器（如 TIM1、TIM8）：除了具有通用定时器的功能外，还具有**输出互补带死区的信号**、**刹车**等功能，主要应用于**电机控制**等需要**高精度**和**复杂控制**的场合。

2.3 定时器的工作原理

时钟源选择：定时器的时钟信号来源有两种，即**内部时钟**和**外部时钟**。内部时钟是由 RCC 的 APB 总线提供的时钟信号，而外部时钟则是通过某个 GPIO 引脚捕获的电平跳变信号。

时基单元：定时器的核心部分包括 **PSC**（预分频器）、**CNT**（计数器）和 **ARR**（自动重装寄存器）。**PSC** 对时钟信号进行分频，**CNT** 在时钟信号的触发下递增或递减计数，当 **CNT** 的值达到 **ARR** 时，触发更新中断或更新事件，并清零 **CNT**。

计数模式：定时器支持三种计数模式，即**向上计数模式**、**向下计数模式**和**中央对齐模式**（**向上/向下模式**）。在不同的计数模式下，定时器的行为会有所不同。

2.4 定时器的配置与使用

开启时钟：在使用定时器之前，需要先开启其时钟。这通常通过 **RCC**（复位和时钟控制）模块来实现。

配置时钟源：选择定时器的时钟源，可以是内部时钟或外部时钟。

时基单元初始化：配置 **PSC**、**CNT** 和 **ARR** 的值，以设置定时器的定时时间和计数模式。

中断配置：如果需要定时器产生中断，则需要配置中断相关的参数，并开启中断使能。

启动定时器：配置完成后，启动定时器开始工作。

2.5 定时器的应用

定时任务：利用定时器产生定时中断，实现定时任务，如**延时**、**周期性任务**等。

测量信号频率和脉冲宽度：通过定时器的输入捕获功能，可以测量外部信号的频率和脉冲宽度。

PWM 输出：利用定时器的 **PWM** 输出功能，可以生成具有特定占空比的 **PWM** 信号，用于**电机控制**、**LED 调光**等应用。

2.6 定时器的例程

HAL 库代码例程

HAL 库（Hardware Abstraction Layer）是 STM32CubeMX 和 STM32CubeIDE 默认使用的软件开发库，提供了更高级别的抽象和更简洁的 API。以下是一个使用 HAL 库配置定时器的简单例程：

```
49 static void MX_TIM2_Init(void) {
50     TIM_ClockConfigTypeDef sClockSourceConfig = {0};
51     TIM_MasterConfigTypeDef sMasterConfig = {0};
52
53     // 使能TIM2时钟
54     __HAL_RCC_TIM2_CLK_ENABLE();
55
56     // 配置TIM2定时器参数
57     htim2.Instance = TIM2;
58     htim2.Init.Prescaler = 7999; // 预分频器值
59     htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
60     htim2.Init.Period = 9999; // 定时器周期
61     htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
```

```
61     htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
62     htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
63     if (HAL_TIM_Base_Init(&htim2) != HAL_OK) {
64         // 初始化错误处理
65         Error_Handler();
66     }
67
68     // 配置时钟源
69     sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
70     if (HAL_TIM_ConfigClockSource(&htim2, &sClockSourceConfig) != HAL_OK) {
71         // 配置时钟源错误处理
72         Error_Handler();
73     }
```

```
74
75     // 配置主模式
76     sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
77     sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
78     if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) != HAL_OK) {
79         // 配置主模式错误处理
80         Error_Handler();
81     }
82 }
83
```

标准库代码例程

以下是一个使用标准库配置定时器的简单例程：

```
C

1  #include "stm32f4xx.h" // 包含具体型号的STM32头文件
2
3  void TIM2_Configuration(void) {
4      TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;
5      NVIC_InitTypeDef NVIC_InitStructure;
6
7      // 使能TIM2和GPIOA时钟
8      RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE);
9      RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA, ENABLE);
10
11     // 配置PA0引脚为输出模式（用于LED）
12     GPIO_InitTypeDef GPIO_InitStruct;
13     GPIO_InitStruct.GPIO_Pin = GPIO_Pin_0;
14     GPIO_InitStruct.GPIO_Mode = GPIO_Mode_OUT;
15     GPIO_InitStruct.GPIO_OType = GPIO_OType_PP;
16     GPIO_InitStruct.GPIO_Speed = GPIO_Speed_50MHz;
17     GPIO_InitStruct.GPIO_PuPd = GPIO_PuPd_NOPULL;
18     GPIO_Init(GPIOA, &GPIO_InitStruct);
19
20     // 配置TIM2定时器
21     TIM_TimeBaseStructure.TIM_Period = 9999; // 定时器周期
22     TIM_TimeBaseStructure.TIM_Prescaler = 7999; // 预分频器值
23     TIM_TimeBaseStructure.TIM_ClockDivision = TIM_CKD_DIV1;
24     TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;
25     TIM_TimeBaseInit(TIM2, &TIM_TimeBaseStructure);
26
```

```

27    // 使能TIM2中断
28    NVIC_InitStructure.NVIC_IRQChannel = TIM2_IRQn;
29    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
30    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
31    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
32    NVIC_Init(&NVIC_InitStructure);
33
34    // 使能TIM2更新中断
35    TIM_ITConfig(TIM2, TIM_IT_Update, ENABLE);
36
37    // 启动TIM2
38    TIM_Cmd(TIM2, ENABLE);
39 }
40

```

2.7 定时器面试常见问题

1. STM32 定时器的种类有哪些？
2. 如何配置 STM32 定时器的时钟源和计数模式？
3. STM32 定时器如何产生中断？
4. 如何利用 STM32 定时器测量信号的频率和脉冲宽度？
5. STM32 定时器的 PWM 输出功能如何应用？

3. GPIO 总结

3.1 GPIO 基本概念

GPIO（General Purpose Input/Output）即通用输入/输出端口，是 STM32 与外部设备交互的关键外设。通过配置 GPIO，MCU 可以控制外部设备的输入和输出信号。

3.2 GPIO 配置

模式配置：使用 GPIOx_MODER 寄存器设置 GPIO 引脚的模式，可以是输入、输出、推挽输出或开漏输出。

速度配置：使用 GPIOx_OSPEEDR 寄存器设置 GPIO 引脚的输出速度，可以是低速、中速或高速。

输出类型配置：使用 GPIOx_OTYPER 寄存器设置 GPIO 引脚的输出类型，可以是推挽输出或开漏输出。

上拉/下拉电阻配置：使用 GPIOx_PUPDR 寄存器设置 GPIO 引脚的上拉或下拉电阻，可以是上拉、下拉或无电阻。

3.3 GPIO 工作模式（8 种，面试喜欢问）

1. **模拟输入（AIN）：**用于检测外部输入的模拟电压。
2. **浮空输入（IN_FLOATING）：**引脚既不配置上拉电阻也不配置下拉电阻，电平状态由外部输入决定。
3. **下拉输入（IPD）：**引脚悬空无输入信号时，输入端的电平可以默认保持在低电平。
4. **上拉输入（IPU）：**引脚悬空无输入信号时，输入端的电平可以默认保持在高电平。
5. **开漏输出（Out_OD）：**需要借助外部上拉电阻才能真正输出高电平，常用于通信接口。
6. **推挽输出（Out_PP）：**可以同时输出高电平和低电平，驱动能力强。
7. **复用开漏输出（AF_OD）：**由片上外设决定输出，常用于通信接口。
8. **复用推挽输出（AF_PP）：**由片上外设决定输出，常用于外设功能引脚。

3.4 GPIO 的例程

基于 HAL 库的 GPIO 代码例程

1. GPIO 初始化

在 HAL 库中，GPIO 的初始化通常通过 STM32CubeMX 工具自动生成，但也可以手动编写。以下是一个手动编写的 GPIO 初始化例程：

```
1  GPIO_InitTypeDef GPIO_InitStructure = {0};
2
3  // 使能GPIO时钟
4  __HAL_RCC_GPIOA_CLK_ENABLE();
5
6  // 配置GPIO引脚为输出模式，推挽输出，高速，无上拉/下拉
7  GPIO_InitStructure.Pin = GPIO_PIN_5;
8  GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_PP;
9  GPIO_InitStructure.Pull = GPIO_NOPULL;
10 GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
11 HAL_GPIO_Init(GPIOA, &GPIO_InitStructure);
```

2. GPIO 读写操作

写操作：使用 HAL_GPIO_WritePin 函数设置 GPIO 引脚的电平状态，如下图：

```
C
1 // 将GPIOA的PA5引脚设置为高电平
2 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
3
4 // 将GPIOA的PA5引脚设置为低电平
5 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
```

读操作：使用 HAL_GPIO_ReadPin 函数读取 GPIO 引脚的电平状态，如下图：

```
1 // 读取GPIOA的PA5引脚电平状态
2 uint8_t pinState = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_5);
3 if (pinState == GPIO_PIN_SET) {
4     // 引脚为高电平
5 } else {
6     // 引脚为低电平
7 }
```

3.GPIO 中断处理

在 HAL 库中，配置 GPIO 中断通常也需要通过 STM32CubeMX 工具进行，但中断服务函数需要手动编写。以下是一个 GPIO 中断处理的例程：

配置中断：在 STM32CubeMX 中配置 GPIO 为外部中断模式，并设置触发方式（上升沿、下降沿或两者都触发）。

编写中断服务函数：在 stm32fxx_it.c 文件中编写中断服务函数。

```

1 // GPIO中断服务函数
2 void EXTI5_IRQHandler(void) {
3     // 检查是否为PA5引脚的中断
4     if (__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_5) != RESET) {
5         // 清除中断标志
6         __HAL_GPIO_EXTI_CLEAR_IT(GPIO_PIN_5);
7
8         // 执行中断处理逻辑
9         // ...
10    }
11 }

```

基于标准库的 GPIO 代码例程

1. GPIO 初始化

在标准库中，GPIO 的初始化需要手动配置寄存器。以下是一个 GPIO 初始化的例程：

```

1 GPIO_InitTypeDef GPIO_InitStructure;
2
3 // 使能GPIO时钟
4 RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);
5
6 // 配置GPIO引脚为输出模式，推挽输出，速度为50MHz
7 GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
8 GPIO_InitStructure.GPIO_Pin = GPIO_Pin_5;
9 GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
10 GPIO_Init(GPIOA, &GPIO_InitStructure);

```

2. GPIO 读写操作

写操作：使用 GPIO_SetBits 或 GPIO_ResetBits 函数设置 GPIO 引脚的电平状态，如下图

```

1 // 将GPIOA的PA5引脚设置为高电平
2 GPIO_SetBits(GPIOA, GPIO_Pin_5);
3
4 // 将GPIOA的PA5引脚设置为低电平
5 GPIO_ResetBits(GPIOA, GPIO_Pin_5);

```

读操作：使用 GPIO_ReadInputDataBit 函数读取 GPIO 引脚的电平状态，如下图：

```

1 // 读取GPIOA的PA5引脚电平状态
2 uint8_t pinState = GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_5);
3 if (pinState == Bit_SET) {
4     // 引脚为高电平
5 } else {
6     // 引脚为低电平
7 }

```

3. GPIO 中断处理

在标准库中，配置 GPIO 中断需要手动配置 NVIC 和 EXTI 寄存器。以下是一个 GPIO 中断处理的例程：

配置中断：配置 NVIC 和 EXTI 寄存器以启用 GPIO 中断，如下图：

```

1 // 配置NVIC中断优先级
2 NVIC_InitTypeDef NVIC_InitStructure;
3 NVIC_InitStructure.NVIC_IRQChannel = EXTI5_IRQn;
4 NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
5 NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
6 NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
7 NVIC_Init(&NVIC_InitStructure);
8
9 // 配置EXTI寄存器以启用PA5引脚的中断
10 EXTI_InitTypeDef EXTI_InitStructure;
11 EXTI_InitStructure.EXTI_Line = EXTI_Line5;
12 EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
13 EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Rising; // 或EXTI_Trigger_Falling、EXTI_Trigger_Both
14 EXTI_InitStructure.EXTI_LineCmd = ENABLE;
15 EXTI_Init(&EXTI_InitStructure);

```

编写中断服务函数：在 stm32fxx_it.c 文件中编写中断服务函数。

```
1 // GPIO中断服务函数
2 void EXTI5_IRQHandler(void) {
3     // 检查是否为PA5引脚的中断
4     if (EXTI_GetITStatus(EXTI_Line5) != RESET) {
5         // 清除中断标志
6         EXTI_ClearITPendingBit(EXTI_Line5);
7
8         // 执行中断处理逻辑
9         // ...
10    }
11 }
```

3.5 GPIO 面试复习建议

1. **理解 GPIO 的配置和使用：**熟悉 GPIO 的寄存器配置和库函数使用，能够根据需要配置 GPIO 的模式、速度、输出类型和上拉/下拉电阻。
2. **掌握 GPIO 的工作模式：**了解各种工作模式的特点和应用场景，能够根据需要选择合适的模式。
3. **了解 GPIO 中断处理：**熟悉 GPIO 中断的配置和中断服务程序的编写，能够处理 GPIO 引脚状态变化时产生的中断。
4. **熟悉寄存器和库函数：**通过查看参考手册和库函数文档，熟悉 GPIO 相关寄存器的位定义和库函数的使用。

4. 串口总结

4.1 USART 与 UART 的对比

USART (Universal Synchronous/Asynchronous Receiver/Transmitter) 是通用同步/异步收发传输器，

而 **UART (Universal Asynchronous Receiver/Transmitter)** 通常被称作通用异步收发传输器。它们主要用于串行通信，但 USART 相比 UART 增加了同步通信的功能。

通信方式：UART 是异步通信，而 USART 可以支持同步和异步两种通信方式。

数据格式：两者在数据格式上相似，都包括起始位、数据位、校验位和停止位。

应用场景：UART 常用于嵌入式系统中的主机与辅助设备之间的通信，而 USART 由于其同步通信的功能，可能在一些需要**更高通信可靠性的场合**中使用。

4.2 UART

基本特性：UART 是串行、全双工、异步通信总线。

数据格式：空闲时，RXD 和 TXD 默认高电平；数据起始位为低电平；串口发送数据过程中，低位在前，高位在后；串口数据一般为 8 位；可选择是否开启数据校验（如奇偶校验）；停止位为高电平，一般有 1、1.5、2 位。

传输速率：波特率表示 1 秒传输数据位数（每秒传送多少 bit）。两个单片机通讯时，波特率需要保持一致以避免数据错误。

硬件连接：采用交叉连接方式，实现点对点通信。

4.3 RS232

定义与特性：RS232 是一种串行通信接口标准，规定了电平值和接口。与 UART 相比，它更侧重于电气特性的定义。

电平标准：-5 到-15V 为逻辑 1（高电平），+5 到+15V 为逻辑 0（低电平）。

通信特性：全双工通信，但传输距离较短（约 15m），易产生共模干扰，抗噪声干扰能力较弱，通信速度相对较低。

接口兼容性：接口的信号电平值较高，易损坏接口电路的芯片，且与 TTL 不兼容，需要使用电平转换芯片。

4.4 RS485

定义与特性：RS485 是一种差分传输的串行通信接口标准，支持多设备通信。

电平标准：电压差为+2 到+6V 为逻辑高电平 1，电压差-2 到-6V 为逻辑低电平 0。

通信特性：半双工通信，但通信速度快，通信距离远（可达 1500m），抗噪声干扰能力强。

接口兼容性：接口的信号电平值较低，不易损坏接口电路的芯片，且与 TTL 兼容，方便

与 TTL 连接。可实现多节点组网（多设备通信，能形成网络）。

4.5 例程

基于 HAL 库的编程例程

1. 初始化串口

首先，需要创建一个 UART_HandleTypeDef 结构体变量，用于存储串口配置和操作的相关信息。然后，配置串口参数，如波特率、数据位、停止位等，并调用 HAL_UART_Init 函数进行初始化，如下图：

```
1  UART_HandleTypeDef huart1;
2
3  void MX_USART1_UART_Init(void)
4  {
5      huart1.Instance = USART1;
6      huart1.Init.BaudRate = 9600;
7      huart1.Init.WordLength = UART_WORDLENGTH_8B;
8      huart1.Init.StopBits = UART_STOPBITS_1;
9      huart1.Init.Parity = UART_PARITY_NONE;
10     huart1.Init.Mode = UART_MODE_TX_RX;
11     huart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;
12     huart1.Init.OverSampling = UART_OVERSAMPLING_16;
13
14     huart1.Init.OneBitSampling = UART_ONE_BIT_SAMPLE_DISABLE;
15     huart1.AdvancedInit.AdvFeatureInit = UART_ADVFEATURE_NO_INIT;
16     if (HAL_UART_Init(&huart1) != HAL_OK)
17     {
18         // Initialization Error
19         Error_Handler();
20     }
21 }
```

在 main 函数中调用 MX_USART1_UART_Init 函数来初始化串口，接上图。

2. 发送数据

使用 HAL_UART_Transmit 函数发送数据。该函数需要传入串口句柄、数据指针和数据长度，如下图：

```

1  uint8_t tData[] = "Hello, STM32!";
2
3  void UART_SendData(uint8_t* pData, uint16_t Size)
4  {
5      if (HAL_UART_Transmit(&huart1, pData, Size, HAL_MAX_DELAY) != HAL_OK)
6      {
7          // Transmission Error
8          Error_Handler();
9      }
10 }

```

在 main 函数中调用 UART_SendData 函数来发送数据，如上图。

3. 接收数据

可以使用 HAL_UART_Receive 函数来接收数据，但更常见的是使用**中断或 DMA**来接收数据，以提高效率。

使用中断接收数据：

首先，调用 HAL_UART_Receive_IT 函数来启动接收中断。然后，在中断服务函数或回调函数中处理接收到的数据，如下图：

```

1  uint8_t rData[10];
2
3  void UART_EnableInterrupt(void)
4  {
5      if (HAL_UART_Receive_IT(&huart1, rData, sizeof(rData)) != HAL_OK)
6      {
7          // Configuration Error
8          Error_Handler();
9      }
10 }

```

```

12 void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
13 {
14     if (huart->Instance == USART1)
15     {
16         // Process received data
17         // rData contains the received data
18         // You can add your own code here to handle the received data
19     }
20
21     // Restart reception in interrupt mode
22     HAL_UART_Receive_IT(huart, rData, sizeof(rData));
23 }

```

在 main 函数中调用 UART_EnableInterrupt 函数来启动接收中断，如上图。

RS232 基于 HAL 库的编程例程

RS232 的编程例程与 USART/UART 类似, 因为 **RS232 通常是通过 USART/UART 接口来实现的**。只需按照上述 USART/UART 的编程步骤进行配置和操作即可。

RS485 基于 HAL 库的编程例程

在使用 STM32 的 RS485 通信时, 需要将 **USART 设置为 RS485 模式**, 并配置 GPIO 来控制 RS485 芯片的发送使能 (**DE**) 和接收使能 (**RE**) 引脚。以下是一个简单的 RS485 编程例程:

```
6  UART_HandleTypeDef huart2;
7
8  void RS485_GPIO_Init(void)
9  {
10     GPIO_InitTypeDef GPIO_InitStruct = {0};
11
12     // Configure DE and RE pins
13     GPIO_InitStruct.Pin = RS485_DE_PIN | RS485_RE_PIN;
14     GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
15     GPIO_InitStruct.Pull = GPIO_NOPULL;
16
17     GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
18     HAL_GPIO_Init(RS485_DE_GPIO, &GPIO_InitStruct);
19
20     // Configure USART TX and RX pins
21     GPIO_InitStruct.Pin = GPIO_PIN_2 | GPIO_PIN_3; // Assuming USART2 on GPIOA
22     GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
23     GPIO_InitStruct.Pull = GPIO_NOPULL;
24     GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
25     GPIO_InitStruct.Alternate = GPIO_AF7_USART2;
26     HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
27 }
```

```

28 void RS485_USART_Init(void)
29 {
30     huart2.Instance = USART2;
31     huart2.Init.BaudRate = 115200;
32     huart2.Init.WordLength = UART_WORDLENGTH_8B;
33     huart2.Init.StopBits = UART_STOPBITS_1;
34     huart2.Init.Parity = UART_PARITY_NONE;
35     huart2.Init.Mode = UART_MODE_TX_RX;
36     huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
37     huart2.Init.OverSampling = UART_OVERSAMPLING_16;
38     huart2.Init.OneBitSampling = UART_ONE_BIT_SAMPLE_DISABLE;
39

```

```

40     if (HAL_UART_Init(&huart2) != HAL_OK)
41     {
42         // Initialization Error
43         Error_Handler();
44     }
45 }

```

1. 发送和接收数据

发送和接收数据的操作与 USART/UART 相同，可以使用 HAL_UART_Transmit 和 HAL_UART_Receive（或中断/DMA 方式）来实现。

2. 控制 RS485 方向

在发送数据之前，将 DE 引脚设置为高电平，RE 引脚设置为低电平；在接收数据之前，将 DE 引脚设置为低电平，RE 引脚设置为高电平。这通常通过 GPIO 操作来实现，代码如下：

```

1 void RS485_SendMode(void)
2 {
3     HAL_GPIO_WritePin(RS485_DE_GPIO, RS485_DE_PIN, GPIO_PIN_SET);
4     HAL_GPIO_WritePin(RS485_RE_GPIO, RS485_RE_PIN, GPIO_PIN_RESET);
5 }
6
7 void RS485_ReceiveMode(void)
8 {
9     HAL_GPIO_WritePin(RS485_DE_GPIO, RS485_DE_PIN, GPIO_PIN_RESET);
10    HAL_GPIO_WritePin(RS485_RE_GPIO, RS485_RE_PIN, GPIO_PIN_SET);
11 }

```

在发送数据之前调用 RS485_SendMode 函数，在接收数据之前调用 RS485_ReceiveMode

函数，如上图。

4.6 面试常见问题

USART 相关问题

USART 的主要特点是什么？

USART 的数据帧格式是怎样的？包括哪些部分？

USART 配置的一般步骤是什么？

STM32 中 USART 的波特率是如何设置的？

如何使用 DMA 与 USART 结合进行数据传输？

UART 相关问题

UART 通信中，波特率、数据位、停止位和校验位的作用是什么？

在 STM32 中，如何配置 UART 的 GPIO 端口模式？

RS232 相关问题

RS232 通信的电气特性是什么？如信号电平和逻辑关系。

RS232 通信中，如何实现数据的全双工传输？

RS232 接口在 STM32 中的应用场景有哪些？

RS485 相关问题

RS485 通信与 RS232 相比有哪些优势？

RS485 通信中，如何实现多机通信和地址识别？

RS485 通信中常见的故障及解决方法有哪些？如数据错乱、通信不稳定等。

在 STM32 中，如何配置 RS485 的 GPIO 端口和通信参数？

如何解决 RS485 通信中的阻抗失配问题？

5. IIC 总结

5.1 I²C 总线基础

1. I²C 总线概述：

I²C（Inter-Integrated Circuit）总线是一种由 Philips 公司开发的用于连接低速外设的串行通信总线。

它具有两根数据线（SDA 和 SCL）和一根地线（GND），可以实现多个设备之间的通信。

2. I²C 总线信号：

开始信号：在 SCL 为高电平时，SDA 从高电平变为低电平，表示开始传输数据。

结束信号：在 SCL 为高电平时，SDA 从低电平变为高电平，表示结束传输数据。

应答信号：接收数据的设备在接收到 8 位数据后，会向发送数据的设备发出一个应答信号（ACK），表示数据已正确接收。（这里注意喔，不是全是从设备发 ACK，是接

收方发 ACK!!!)

3. I²C 总线的工作模式:

标准模式: 最高速率为 100kbps。

快速模式: 最高速率为 400kbps。

快速模式增强版: 最高速率为 1.7Mbps (部分 STM32 型号可能支持更高的速率, 但需注意具体型号规格)。

高速模式: 理论上最高速率可达 3.4Mbps, 但并非所有 STM32 型号都支持此模式。

(注意喔: bps 是每秒多少 bit, 不是多少 byte, 因为是串行通信, 是一个 bit 一个 bit 的传数据的!!! 不少同学面试模拟答成 byte, NG)

5.2 STM32 中的 I²C 通信

1. I²C 接口配置:

在 STM32 中, I²C 接口通常通过配置相应的寄存器来实现。

需要配置 I²C 的时钟源、速率、地址模式、从设备地址等参数。

2. I²C 通信过程: (面试高频问点: 通信流程必须理解背诵, 不能漏地址和读写位!!!)

主机发送开始信号, 然后发送从设备地址和读写位。

从设备接收到地址后, 如果匹配则发出应答信号。

主机发送或接收数据, 每发送或接收 8 位数据后, 从设备都会发出应答信号。

通信结束后, 主机发送结束信号。

3. I²C 中断和 DMA 支持:

STM32 的 I²C 接口支持中断和 DMA 功能, 可以实现数据的异步传输和高速数据传输。

通过配置中断优先级和中断服务函数, 可以处理 I²C 通信中的各种事件, 如接收完成、发送完成等。

DMA 功能可以实现数据的自动传输, 减轻 CPU 的负担。

5.3 I²C 在 32 的例程

I²C HAL 库例程

使用 STM32 HAL 库进行 I²C 通信的初始化及基本读写操作。

1. 初始化 I²C 外设, 如下图:

```

1  I2C_HandleTypeDef hi2c1;
2
3  void MX_I2C1_Init(void)
4  {
5      hi2c1.Instance = I2C1;
6      hi2c1.Init.ClockSpeed = 100000; // 设置I2C时钟速度为100kHz
7      hi2c1.Init.DutyCycle = I2C_DUTYCYCLE_2;
8      hi2c1.Init.OwnAddress1 = 0;
9      hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
10     hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
11     hi2c1.Init.OwnAddress2 = 0;
12     hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
13     hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
14     if (HAL_I2C_Init(&hi2c1) != HAL_OK)
15     {
16         // 初始化错误处理
17         Error_Handler();
18     }
19 }

```

2. I²C 读写操作，如下图：

```

1  // 写操作
2  HAL_StatusTypeDef HAL_I2C_Write(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint8_t
3  {
4      return HAL_I2C_Master_Transmit(hi2c, DevAddress, pData, Size, Timeout);
5  }
6
7  // 读操作
8  HAL_StatusTypeDef HAL_I2C_Read(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint8_t
9  {
10     return HAL_I2C_Master_Receive(hi2c, DevAddress, pData, Size, Timeout);
11 }

```

补充：

```

2  typedef struct {
3      I2C_HandleTypeDef *hi2c;
4      uint16_t DevAddress;
5      uint8_t *pData;
6      uint16_t Size;
7      uint32_t Timeout;
8  } I2C_HandleTypeDef;

```

```

8  typedef struct {
9      I2C_HandleTypeDef *hi2c;
10     uint16_t DevAddress;
11     uint8_t *pData;
12     uint16_t Size;
13     uint32_t Timeout;
14 } I2C_HandleTypeDef;

```

I²C 标准库例程

（注意不同的 STM32 型号和标准库版本可能会有不同的函数和宏定义！！）

使用 STM32 标准库进行 I²C 通信的初始化及基本读写操作。

1. GPIO 初始化，如下图：

```
1 void I2C_GPIO_Config(void)
2 {
3     GPIO_InitTypeDef GPIO_InitStructure;
4
5     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);
6
7     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6 | GPIO_Pin_7; // SDA, SCL
8     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
9     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
10    GPIO_Init(GPIOB, &GPIO_InitStructure);
11
12    GPIO_SetBits(GPIOB, GPIO_Pin_6 | GPIO_Pin_7); // 初始化SCL和SDA为高电平
13 }
```

2. I²C 初始化，如下图：

```
1 void I2C_Config(void)
2 {
3     I2C_InitTypeDef I2C_InitStructure;
4
5     RCC_APB1PeriphClockCmd(RCC_APB1Periph_I2C1, ENABLE);
6
7     I2C_InitStructure.I2C_ClockSpeed = 100000; // 设置I2C时钟速度为100kHz
8     I2C_InitStructure.I2C_Mode = I2C_Mode_I2C;
9     I2C_InitStructure.I2C_DutyCycle = I2C_DutyCycle_2;
10    I2C_InitStructure.I2C_OwnAddress1 = 0x00;
11    I2C_InitStructure.I2C_Ack = I2C_Ack_Enable;
12    I2C_InitStructure.I2C_AcknowledgedAddress = I2C_AcknowledgedAddress_7bit;
13    I2C_Init(I2C1, &I2C_InitStructure);
14
15    I2C_Cmd(I2C1, ENABLE);
16 }
```

3. I²C 的读，如下图：

```

40 // I2C写操作
41 uint8_t I2C_WriteData(uint8_t RegAddr, uint8_t Data)
42 {
43     uint8_t status;
44
45     // 生成起始条件
46     I2C_GenerateSTART(I2C1, ENABLE);
47
48     // 等待起始条件生成完成
49     while (!I2C_CheckEvent(I2C1, I2C_EVENT_MASTER_MODE_SELECT));
50
51     // 发送从设备地址和写命令
52     I2C_Send7bitAddress(I2C1, I2C_SLAVE_ADDRESS << 1, I2C_Direction_Transmitter);
53
54     // 等待从设备地址发送完成
55     while (!I2C_CheckEvent(I2C1, I2C_EVENT_MASTER_TRANSMITTER_MODE_SELECTED));
56
57     // 发送寄存器地址
58     I2C_SendData(I2C1, RegAddr);
59
60     // 等待寄存器地址发送完成
61     while (!I2C_CheckEvent(I2C1, I2C_EVENT_MASTER_BYTE_TRANSMITTED));
62
63     // 发送数据
64     I2C_SendData(I2C1, Data);
65
66     // 等待数据发送完成
67     while (!I2C_CheckEvent(I2C1, I2C_EVENT_MASTER_BYTE_TRANSMITTED));
68
69     // 生成停止条件
70     I2C_GenerateSTOP(I2C1, ENABLE);
71
72     // 检查通信状态
73     if (I2C_GetErrorCode(I2C1) != I2C_ERROR_NONE)
74     {
75         status = 0xFF; // 通信错误
76     }
77     else
78     {
79         status = 0x00; // 通信成功
80     }
81
82     return status;
83 }

```

4. I²C 的写，如下图:

```

85 // I2C读操作
86 uint8_t I2C_ReadData(uint8_t RegAddr)
87 {
88     uint8_t Data;
89
90     // 生成起始条件
91     I2C_GenerateSTART(I2C1, ENABLE);
92
93     // 等待起始条件生成完成
94     while (!I2C_CheckEvent(I2C1, I2C_EVENT_MASTER_MODE_SELECT));
95
96     // 发送从设备地址和写命令
97     I2C_Send7bitAddress(I2C1, I2C_SLAVE_ADDRESS << 1, I2C_Direction_Transmitter);
98
99     // 等待从设备地址发送完成
100    while (!I2C_CheckEvent(I2C1, I2C_EVENT_MASTER_TRANSMITTER_MODE_SELECTED));
101
102    // 发送寄存器地址
103    I2C_SendData(I2C1, RegAddr);
104

```

```

104
105    // 等待寄存器地址发送完成
106    while (!I2C_CheckEvent(I2C1, I2C_EVENT_MASTER_BYTE_TRANSMITTED));
107
108    // 生成重复起始条件以切换为读模式
109    I2C_GenerateSTART(I2C1, ENABLE);
110
111    // 等待重复起始条件生成完成
112    while (!I2C_CheckEvent(I2C1, I2C_EVENT_MASTER_MODE_SELECT));
113
114    // 发送从设备地址和读命令
115    I2C_Send7bitAddress(I2C1, I2C_SLAVE_ADDRESS << 1, I2C_Direction_Receiver);
116
117    // 等待从设备地址接收完成
118    while (!I2C_CheckEvent(I2C1, I2C_EVENT_MASTER_RECEIVER_MODE_SELECTED));
119
120    // 接收数据
121    if (I2C_WaitOnFlagUntilTimeout(I2C1, I2C_FLAG_RXNE, SET, I2C_TIMEOUT_FLAG) != HAL
122    {

```

```

123     // 超时错误处理
124     return 0xFF;
125 }
126 Data = I2C_ReceiveData(I2C1);
127
128 // 生成停止条件
129 I2C_GenerateSTOP(I2C1, ENABLE);
130
131 return Data;
132 }

```

注意事项:

1. I²C_TIMEOUT_FLAG 是一个超时标志的宏定义，你需要根据你的具体需求和系统时钟来设置它。在标准库中，这个宏可能需要你自己定义，因为它不是标准库的一部分。
2. I2C_WaitOnFlagUntilTimeout 函数在标准库中可能不存在，或者其名称和功能可能有所不同。这个函数的作用是等待一个特定的标志被设置，如果在指定的时间内标志没有被设置，则返回超时错误。在标准库中，你可能需要使用轮询和延时函数来实现类似的功能。
3. 在实际应用中，你可能需要添加更多的错误处理和状态检查代码来确保 I²C 通信的可靠性。
4. 确保你的 STM32 型号和标准库版本与上述代码兼容。不同的 STM32 型号和标准库版本可能会有不同的函数和宏定义。

I²C 与 DMA 结合的例程，基于 HAL 库

使用 DMA 来加速 I²C 的数据传输。

1. 初始化 DMA，如下图：

```

1 DMA_HandleTypeDef hdma_i2c_tx;
2
3 void MX_DMA_Init(void)
4 {
5     __HAL_RCC_DMA1_CLK_ENABLE();
6
7     hdma_i2c_tx.Instance = DMA1_Channel4; // 根据具体的I2C外设选择DMA通道
8     hdma_i2c_tx.Init.Direction = DMA_MEMORY_TO_PERIPH;
9     hdma_i2c_tx.Init.PeriphInc = DMA_PINC_DISABLE;
10    hdma_i2c_tx.Init.MemInc = DMA_MINC_ENABLE;
11    hdma_i2c_tx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE;
12    hdma_i2c_tx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE;
13    hdma_i2c_tx.Init.Mode = DMA_NORMAL;
14    hdma_i2c_tx.Init.Priority = DMA_PRIORITY_HIGH;
15    hdma_i2c_tx.Init.FIFOMode = DMA_FIFOMODE_DISABLE;
16    if (HAL_DMA_Init(&hdma_i2c_tx) != HAL_OK)
17    {
18        // 初始化错误处理
19        Error_Handler();
20    }

```

2. 配置 I²C 以使用 DMA，如下图：

```

1 void I2C_DMA_Config(void)
2 {
3     // 假设已经初始化了I2C1和DMA1_Channel4
4     // 设置I2C1的TX DMA请求
5     hi2c1.HDMATx = &hdma_i2c_tx;
6     hi2c1.hdmарx = NULL; // 如果不使用RX DMA，则设置为NULL
7
8     // 其他I2C配置...
9 }

```

3.启动 DMA 传输，如下图：

```

1 void I2C_DMA_Transmit(uint16_t DevAddress, uint8_t *pData, uint16_t Size)
2 {
3     // 配置DMA传输参数
4     HAL_DMA_Start(&hdma_i2c_tx, (uint32_t)pData, (uint32_t)&I2C1->DR, Size);
5
6     // 启动I2C传输，并启用DMA
7     if (HAL_I2C_Master_Transmit_DMA(&hi2c1, DevAddress, pData, Size, HAL_MAX_DELAY) !=
8     {
9         // 传输错误处理
10        Error_Handler();
11    }
12 }

```

注意事项（面试的时候可能会问需要注意什么！！！）

1. 在使用 DMA 时，需要**确保 DMA 通道和 I²C 外设之间的正确连接**。
2. 在配置 I²C 和 DMA 时，要注意**时钟使能、中断优先级、数据对齐方式等参数**的设置。
3. 在实际项目中，可能要根据具体的硬件和需求对代码进行调整和优化。

5.4 I²C 通信的应用场景和注意事项

1. 应用场景：

I²C 总线常用于**连接低速外设**，如传感器、**存储器（EEPROM）**、显示器等。

在 STM32 的应用中，I²C 总线可以用于读取传感器的数据、与存储器进行数据传输等。

2. 注意事项：

在进行 I²C 通信时，需要注意**设备的地址分配**，避免地址冲突。

需要根据设备的特性选择合适的**通信速率和工作模式**。

在使用 DMA 功能时，需要正确配置**DMA 通道和传输参数**。

5.5 面试中可能遇到的问题（下面 4 点常问务必拿下！！！）

1. 描述 I²C 总线的基本工作原理。
2. 如何在 STM32 中配置 I²C 接口？
3. I²C 通信过程中可能出现的问题及解决方法。
4. 如何使用 DMA 与 I²C 结合进行数据传输？

6. SPI 总结

6.1 SPI 基本概念

SPI（Serial Peripheral Interface）是由美国摩托罗拉公司最先推出的一种同步全双工串行传输规范，也是一种单片机外设芯片串行扩展接口。SPI 具有**高速、全双工、串行和同步**等特点。

6.2 SPI 通信流程（务必理解背诵！！！）

SPI 的通信流程可以分为以下四个步骤：

1. **主机进行片选**：主设备通过**拉低 CS/SS 信号**，启动与从设备的通信。
2. **发送时钟信号**：主设备发送时钟信号，告诉从设备进行写数据或读数据操作。时钟信号的有效边沿（上升沿或下降沿）**取决于 SPI 的工作模式**。

3. **主机收发数据：**主机将要发送的数据写到发送数据缓存区，然后通过 MOSI 信号线将字节一位一位地移出去传送给从机。同时，主机通过 MISO 信号线接收从机发送的数据。

4. **从机收发数据：**从机也将自己的数据通过 MISO 信号线返回给主机，同时通过 MOSI 信号线接收主机发送的数据。

6.3 SPI 工作模式

SPI 有四种不同的工作模式，这些模式由时钟极性（CPOL）和时钟相位（CPHA）的组合决定：

Mode 0： CPOL=0，CPHA=0。空闲时，时钟线为低电平；数据在第一个时钟边沿（上升沿）被采样。

Mode 1： CPOL=0，CPHA=1。空闲时，时钟线为低电平；数据在第二个时钟边沿（下降沿）被采样。

Mode 2： CPOL=1，CPHA=0。空闲时，时钟线为高电平；数据在第一个时钟边沿（下降沿）被采样。

Mode 3： CPOL=1，CPHA=1。空闲时，时钟线为高电平；数据在第二个时钟边沿（上升沿）被采样。

6.4 SPI 的优缺点（务必理解背诵！！！）

优点：

无起始位和停止位，数据位可以连续传输而不会被中断。

没有像 I2C 那样复杂的从设备寻址系统。

数据传输速率比 I2C 更高。

分离的 MISO 和 MOSI 信号线，可以同时发送和接收数据。

硬件结构简单，从机不需要唯一地址。

缺点：

使用四根信号线（I2C 和 UART 使用两根信号线）。

无法确认是否已成功接收数据（I2C 拥有此功能）。

没有任何形式的错误检查（如 UART 中的奇偶校验位）。

只允许一个主设备。

6.5 STM32 中 SPI 控制器的配置

在 STM32 中，SPI 控制器的配置包括 GPIO 引脚的配置和 SPI 控制器的配置：

GPIO 引脚配置：

- 1.使能 GPIO 时钟。
- 2.初始化配置 GPIO，完成 GPIO 的四个配置寄存器的赋值。
- 3.配置具体的复用功能。

SPI 控制器配置：

- 1.使能 SPI 时钟。
- 2.配置 SPI 控制器，包括设置 SPI 的通信方式（全双工/半双工等）、主从模式、数据位长度、
- 3.时钟极性和时钟相位等。

6.6 SPI 的例程：

基于 HAL 库的 SPI 编程

1. 配置 SPI

使用 STM32CubeMX 或 STM32CubeIDE 进行 SPI 配置，包括选择 SPI 接口、设置工作模式、配置时钟极性和相位、设置数据大小等。

生成初始化代码后，在 `MX_SPIx_Init()` 函数中可以看到 SPI 的具体配置。

2. 发送和接收数据

使用 `HAL_SPI_Transmit()` 函数发送数据。

使用 `HAL_SPI_Receive()` 函数接收数据。

也可以使用 `HAL_SPI_TransmitReceive()` 函数同时发送和接收数据。

3. 中断和 DMA

可以配置 SPI 中断来处理数据传输完成等事件。

可以使用 DMA（Direct Memory Access）来加速数据传输，减少 CPU 负担。

在中断或 DMA 回调函数中可以处理接收到的数据或准备发送的数据。

```

4  SPI_HandleTypeDef hspi1;
5
6  // SPI1 初始化函数
7  void MX_SPI1_Init(void)
8  {
9      // SPI 配置结构体初始化
10     hspi1.Instance = SPI1; // 指定SPI实例为SPI1
11     hspi1.Init.Mode = SPI_MODE_MASTER; // 配置为主模式
12     hspi1.Init.Direction = SPI_DIRECTION_2LINES; // 全双工模式
13     hspi1.Init.DataSize = SPI_DATASIZE_8BIT; // 数据大小为8位
14     hspi1.Init.CLKPolarity = SPI_POLARITY_LOW; // 时钟极性为低 (CPOL=0)
15     hspi1.Init.CLKPhase = SPI_PHASE_1EDGE; // 时钟相位为第一个边沿 (CPHA=0)，即数据在上边沿采样
16     hspi1.Init.NSS = SPI_NSS_SOFT; // 使用软件NSS (片选信号)，不使用硬件NSS引脚
17     hspi1.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_16; // 时钟预分频值，设置SPI时钟频率为系统时钟的1/16
18     hspi1.Init.FirstBit = SPI_FIRSTBIT_MSB; // 数据传输从MSB (最高位) 开始
19     hspi1.Init.TIMode = SPI_TIMODE_DISABLE; // 禁用TI模式 (用于同步串行接口)
20     hspi1.Init.CRCCalculation = SPI_CRCCALCULATION_DISABLE; // 禁用CRC计算
21     hspi1.Init.CRCPolynomial = 10; // CRC多项式 (如果CRC计算启用)
22
23     // 如果需要，可以配置SPI中断和DMA (这里未展示)

```

```

24
25     // 初始化SPI外设
26     if (HAL_SPI_Init(&hspi1) != HAL_OK)
27     {
28         // 初始化错误处理
29         Error_Handler(); // 这是一个自定义的错误处理函数，你需要自己实现
30     }
31 }

```

```

1  // 初始化SPI
2  MX_SPI1_Init();
3
4  // 发送数据
5  uint8_t txData[] = {0x01, 0x02, 0x03};
6  HAL_SPI_Transmit(&hspi1, txData, sizeof(txData), HAL_MAX_DELAY);
7
8  // 接收数据
9  uint8_t rxData[3];
10 HAL_SPI_Receive(&hspi1, rxData, sizeof(rxData), HAL_MAX_DELAY);
11
12 // 同时发送和接收数据
13 HAL_SPI_TransmitReceive(&hspi1, txData, rxData, sizeof(txData), HAL_MAX_DELAY);

```

基于标准库的 SPI 编程

1. 配置 SPI

在标准库中，需要手动配置 SPI 相关的 GPIO、时钟、NVIC 等。

使用 SPI_InitTypeDef 结构体来配置 SPI 参数。

调用 SPI_Init()函数来初始化 SPI。

2. 发送和接收数据

使用 SPI_SendData()函数发送数据。

使用 SPI_ReceiveData()函数接收数据。

需要轮询 SPI 状态或使用中断来判断数据传输是否完成。

3. 例程代码

```
4  SPI_HandleTypeDef hspi1;
5
6  // SPI1初始化函数
7  void MX_SPI1_Init(void)
8  {
9      // SPI配置结构体初始化
10     hspi1.Instance = SPI1; // 指定要初始化的SPI外设实例为SPI1
11
12     // 配置SPI的工作模式
13     hspi1.Init.Mode = SPI_MODE_MASTER; // 设置为SPI主模式
14     hspi1.Init.Direction = SPI_DIRECTION_2LINES; // 设置为全双工通信模式
15     hspi1.Init.DataSize = SPI_DATASIZE_8BIT; // 设置数据帧大小为8位
16     hspi1.Init.CLKPolarity = SPI_POLARITY_LOW; // 设置时钟极性为低电平有效 (CPOL=0)
17     hspi1.Init.CLKPhase = SPI_PHASE_1EDGE; // 设置时钟相位为第一个时钟边沿采样 (CPHA=0)
18     hspi1.Init.NSS = SPI_NSS_SOFT; // 使用软件NSS信号，不使用硬件NSS引脚
19     hspi1.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_16; // 设置SPI时钟预分频值，
20     hspi1.Init.FirstBit = SPI_FIRSTBIT_MSB; // 设置数据传输从最高位开始
21     hspi1.Init.TIMode = SPI_TIMODE_DISABLE; // 禁用TI模式 (用于同步串行接口)
22     hspi1.Init.CRCCalculation = SPI_CRCCALCULATION_DISABLE; // 禁用CRC校验计算
23     hspi1.Init.CRCPolynomial = 7; // 设置CRC多项式 (如果CRC计算启用)
24
25     // 如果需要，可以配置SPI的中断和DMA (这里未展示)
26
27     // 初始化SPI外设
28     if (HAL_SPI_Init(&hspi1) != HAL_OK)
```

```
14 // 发送数据
15 uint8_t txData = 0x01;
16 SPI_SendData(SPI1, txData);
17 while (SPI_GetFlagStatus(SPI1, SPI_FLAG_TXE) == RESET); // 等待发送完成
18
19 // 接收数据
20 uint8_t rxData;
21 while (SPI_GetFlagStatus(SPI1, SPI_FLAG_RXNE) == RESET); // 等待接收完成
22 rxData = SPI_ReceiveData(SPI1);
```

6.7 常见问题（务必理解背诵！！！）

1. SPI 需要几根线？
2. SPI 的通信速率受什么影响？
3. 和 IIC 的区别？

7. CAN 总结

7.1 CAN 总线基本概念

1. **物理特性：**CAN 总线是一种差分信号传输总线，通过 **CAN_H** 和 **CAN_L** 两根线传输信号，根据两根线上的电位差来判断总线电平。
2. **帧类型：**CAN 帧有五种类型，包括**数据帧**、**远程帧**、**错误帧**、**过载帧**和**帧间隔**。
3. **ACK 应答机制：**用于确认数据是否正常接收，发送节点将此位设为 1，接收节点正常接收数据后将此位置为 0。（类似 IIC 的 ACK）
4. **仲裁机制：**当多个单元同时开始发送时，各发送单元从仲裁段的第一位开始进行仲裁，连续输出显性电平最多的单元可继续发送。

7.2 STM32 中 CAN 的特性

1. **支持协议：**STM32 的 CAN 控制器支持 CAN 协议 2.0A 和 2.0B。
2. **波特率设置：**STM32 单片机的 CAN 波特率主要由**重新同步跳跃时间单元（tsjw）**、**时间段 1 的时间单元（tbs1）**、**时间段 2 的时间单元（tbs2）**和**波特率分频器（brp）**四个参数共同决定。

3. **过滤器：**CAN 过滤器有四种工作模式，包括 32 位宽的掩码模式、32 位宽的列表模式、16 位宽的列表模式和 16 位宽的掩码模式。
4. **工作模式：**STM32 的 CAN 控制器有多种工作模式，如正常模式、环回模式、静默模式和静默环回模式。

7.3 CAN 配置方法

1. **引脚配置：**STM32 的 CAN 物理引脚脚位可以设置成默认模式、重定义地址 1 模式和重定义地址 2 模式。在配置时，需要设置相应的 GPIO 引脚为 CAN 功能。
2. **时钟使能：**在配置 CAN 之前，需要使能 CAN 的时钟。
3. **波特率设置：**根据实际需求，设置 CAN 的波特率参数，包括 tsjw、tbs1、tbs2 和 brp。
4. **过滤器设置：**根据需求设置 CAN 过滤器的模式和 ID，以过滤掉不需要接收的数据帧。

7.4 CAN 的例程：

HAL 库例程

1. CAN 初始化

定义一个 CAN 句柄结构体（例如 CAN_HandleTypeDef hcan1），并在其成员中存储 CAN 的设置参数。

对该结构体的成员进行赋值，包括实例（Instance）、预分频器（Prescaler）、工作模式（Mode）等。

调用 HAL_CAN_Init(&hcan1)函数进行 CAN 的初始化。

2. CAN 引脚初始化

在 HAL_CAN_MspInit 函数中设置 CAN 的引脚模式及中断。

此函数通常在 HAL_CAN_Init 中被调用，具体调用方式取决于宏定义 USE_HAL_CAN_REGISTER_CALLBACKS 的值。

3. 发送和接收数据

使用 HAL_CAN_AddTxMessage 函数发送 CAN 通信帧。

接收数据时，可以通过配置接收中断并在中断回调函数中处理接收到的数据。

```
1 // 发送数据
2 HAL_StatusTypeDef HAL_CAN_AddTxMessage(CAN_HandleTypeDef* hcan, CAN_TxHeaderTypeDef*
3
4 // 接收数据回调函数
5 void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan) {
6     uint8_t Data[8];
7     CAN_RxHeaderTypeDef RxHeader;
8     if (HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &RxHeader, Data) == HAL_OK) {
9         // 处理接收到的数据
10    }
11 }
```

补充：

```
2 .TypeDef* hcan, CAN_TxHeaderTypeDef* pHeader, uint8_t* pData, uint32_t* pTxMailbox);
```

标准库例程

标准库与 HAL 库在配置和使用上有所不同，但基本原理相似。以下是一个标准库配置 CAN 总线的简要步骤：

1. 时钟配置

配置 RCC 时钟，使能 CAN 外设的时钟。

```
1 RCC_APB1PeriphClockCmd(RCC_APB1Periph_CAN1, ENABLE); // 开启CAN1时钟
```

2. GPIO 配置

配置 CAN 总线的 TX 和 RX 引脚为复用功能输出和输入模式。

```
1 GPIO_InitTypeDef GPIO_InitStructure;
2
3 // 配置CAN1的TX引脚（例如PA9）
4 GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;
5 GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP; // 复用推挽输出
6 GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
7 GPIO_Init(GPIOA, &GPIO_InitStructure);
8
9 // 配置CAN1的RX引脚（例如PA8）
10 GPIO_InitStructure.GPIO_Pin = GPIO_Pin_8;
11 GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING; // 浮空输入，对于CAN接收通常配置
12 GPIO_Init(GPIOA, &GPIO_InitStructure);
```

3.CAN 配置

定义一个 CAN 结构体，并对其进行初始化，包括设置波特率、过滤器等。
调用 CAN 初始化函数进行初始化。

```
1 CAN_InitTypeDef CAN_InitStructure;
2 CAN_DeInit(CAN1); // 复位CAN1
3
4 // 配置CAN参数
5 CAN_InitStructure.CAN_TTCM = DISABLE; // 禁止时间触发通信模式
6 CAN_InitStructure.CAN_ABOM = DISABLE; // 禁止自动总线关闭模式
7 CAN_InitStructure.CAN_AWUM = DISABLE; // 禁止自动唤醒模式
8 CAN_InitStructure.CAN_NART = DISABLE; // 禁止报文自动重传
9 CAN_InitStructure.CAN_RFLM = DISABLE; // 禁止FIFO锁定模式
10 CAN_InitStructure.CAN_TXFP = DISABLE; // 禁止发送优先级标志
11 CAN_InitStructure.CAN_Mode = CAN_Mode_Normal; // 正常模式
12
13 // 设置波特率（例如500kbps）
14 CAN_InitStructure.CAN_SJW = CAN_SJW_1tq; // 重新同步跳跃宽度为1个时间单位
15 CAN_InitStructure.CAN_BS1 = CAN_BS1_6tq; // 时间段1为6个时间单位
16 CAN_InitStructure.CAN_BS2 = CAN_BS2_1tq; // 时间段2为1个时间单位
17 CAN_InitStructure.CAN_Prescaler = 9; // 波特率预分频器为9
18
19 CAN_Init(CAN1, &CAN_InitStructure); // 初始化CAN1
```

4.过滤器配置

```
1 CAN_FilterInitTypeDef CAN_FilterInitStructure;
2
3 // 配置过滤器
4 CAN_FilterInitStructure.CAN_FilterNumber = 0; // 过滤器编号
5 CAN_FilterInitStructure.CAN_FilterMode = CAN_FilterMode_IdMask; // 标识符掩码模式
6 CAN_FilterInitStructure.CAN_FilterScale = CAN_FilterScale_32bit; // 32位过滤器
7 CAN_FilterInitStructure.CAN_FilterIdHigh = 0x0000; // 标识符高16位
8 CAN_FilterInitStructure.CAN_FilterIdLow = 0x0000; // 标识符低16位
9 CAN_FilterInitStructure.CAN_FilterMaskIdHigh = 0x0000; // 掩码高16位
10 CAN_FilterInitStructure.CAN_FilterMaskIdLow = 0x0000; // 掩码低16位
11 CAN_FilterInitStructure.CAN_FilterFIFOAssignment = CAN_Filter_FIFO0; // 分配到FIFO0
12 CAN_FilterInitStructure.CAN_FilterActivation = ENABLE; // 激活过滤器
13
14 CAN_FilterInit(&CAN_FilterInitStructure); // 初始化过滤器
```

5. 发送和接收数据

使用标准库提供的发送和接收函数进行数据的发送和接收。

```
1 // 发送数据
2 CanTxMsg TxMessage;
3 TxMessage.StdId = 0x123; // 标准标识符
4 TxMessage.ExtId = 0x00; // 扩展标识符（未使用）
5 TxMessage.RTR = CAN_RTR_DATA; // 数据帧
6 TxMessage.IDE = CAN_ID_STD; // 标准帧
7 TxMessage.DLC = 8; // 数据长度码（8字节数据）
8 TxMessage.Data[0] = 0x01; // 数据字节0
9 TxMessage.Data[1] = 0x02; // 数据字节1
10 // ... 依此类推，填充其他数据字节
11
12 CAN_Transmit(CAN1, &TxMessage); // 发送数据
13
14 // 接收数据（通常通过中断处理）
15 // 在中断服务程序中调用CAN_Receive函数处理接收到的数据
```

7.5 面试中可能遇到的问题（理解去背诵喔！！！）

1. CAN 总线如何判断总线电平？

回答：CAN 控制器根据 CAN_H 和 CAN_L 上的电位差来判断总线电平。

2. CAN 总线的波特率如何设置？

回答：STM32 单片机的 CAN 波特率主要由重新同步跳跃时间单元（tsjw）、时间段 1 的时间单元（tbs1）、时间段 2 的时间单元（tbs2）和波特率分频器（brp）四个参数共同决定。在配置时，需要设置这四个参数的值。

3. CAN 过滤器的作用是什么？如何配置？

回答：CAN 过滤器用于过滤掉不需要接收的数据帧。在配置时，需要设置过滤器的模式和 ID。

4. STM32 的 CAN 控制器支持哪些工作模式？

回答：STM32 的 CAN 控制器支持正常模式、环回模式、静默模式和静默环回模式。

5. CAN 总线的仲裁机制是如何工作的？

回答：当多个单元同时开始发送时，各发送单元从仲裁段的第一位开始进行仲裁，连续输出显性电平最多的单元可继续发送。

6. 在面试中可以从协议本质与通信方式、物理层特性、报文结构与标识符、应用与成本以及 STM32 中的实现等方面来阐述 STM32 中 CAN 和 LIN 的区别。

（无论回答哪个知识点，面试问区别的时候要说到两到三个点，发现学员面试模拟只说 1 个点是不够的！！！）

8. DMA 总结

8.1 DMA 基本概念

1. 定义：DMA，全称 **Direct Memory Access**，即直接存储器访问。它是一种在**外设与存储器之间以及存储器与存储器之间提供高速数据传输的机制**。
2. 作用：通过 DMA，数据可以在无需 CPU 操作的情况下快速移动，从而**节省 CPU 资源**以供其他操作使用。

8.2 DMA 工作原理

1. **数据传输**：DMA 控制器基于复杂的总线矩阵架构，将功能强大的双 AHB 主总线架构与独立的 FIFO 结合在一起，优化了系统带宽。它可以在外设与存储器之间执行数据传输，包括外设到存储器的传输、存储器到外设的传输以及存储器到存储器的传输。
2. **请求与响应**：在产生事件后，外设会向 DMA 控制器发送请求信号。DMA 控制器根据通道的优先级处理该请求，一旦开始访问发出请求的外设，DMA 控制器就会向外设发送确认信号。外设收到确认信号后，会立即释放其请求。一旦外设使请求失效，DMA 控制器就会释放确认信号。
3. **寄存器配置**：DMA 传输的参数（如**源地址、目标地址、传输数据量**等）通过寄存器进行配置。例如，通过 DMA_SxPAR（外设）或 DMA_SxM0AR（存储器）寄存器寻址，可以从外设数据寄存器或存储器单元中加载数据，并将其存储到目标地址。

8.3 DMA 传输模式与特点

1. 传输模式：

正常模式（DMA_Mode_Normal）：每个事务完成后，包含要传输的数据项总量的寄存器都会**递减**。

循环模式（DMA_Mode_Circular）：在到达传输终点时会**重新启动 DMA 传输**，适用于需要连续采集数据的场景，如 GPS、蓝牙等。

2. 特点:

支持多个数据通道和数据流，每个通道都可以配置为特定的外设地址。

支持可编程的数据传输数目，最大为 65535。

支持循环的缓冲器管理，方便处理连续数据流。

每个通道都有事件标志（如 DMA 半传输、DMA 传输完成和 DMA 传输出错），可以触发中断请求。

8.4 DMA 配置与应用

1. 配置步骤:

确定数据传输的源地址和目标地址。

设置数据传输量。

选择传输模式（正常模式或循环模式）。

配置 DMA 控制器的寄存器，包括 DMA_SxCR（控制寄存器）、DMA_SxPAR（外设地址寄存器）、DMA_SxM0AR（存储器地址寄存器）和 DMA_SxNDTR（数据传输量寄存器）等。

2. 应用场景:

数据采集与存储：如 ADC 采集的数据通过 DMA 传输到 SRAM 中。

、

数据通信：如 SPI、I2C 等外设之间的数据传输。

音频/视频处理：如音频数据的实时采集与处理。

8.5 面试常见问题与解答

1. 什么是 DMA?

2.DMA 传输模式有几种?

3.如何配置 DMA 控制器以实现数据传输?

解答：配置 DMA 控制器需要设置数据传输的源地址、目标地址、传输量以及传输模式等参数，并通过寄存器进行配置。具体步骤包括确定数据传输的源和目标、设置传输参数、选择传输模式以及配置 DMA 控制器的相关寄存器。

9.ADC/DAC 总结

9.1 ADC（模数转换器）

1.定义与作用

ADC（Analog-Digital Converter）模拟-数字转换器，可以将引脚上连续变化的模拟电压转换为内存中存储的数字变量，建立模拟电路到数字电路的桥梁。

2.工作原理

采样：在某一时刻采样模拟信号，将连续的模拟信号转换成离散的样本点。

保持：保持采样点的电压，使其在整个转换过程中保持稳定，以避免信号变化带来的误差。

量化：将模拟信号的值映射到一定比例的离散的数字值上。

编码：将量化后的数字转换成二进制数，便于后续的数字处理和存储。

3.STM32 ADC 特点

12 位逐次逼近型 ADC，转换速度快，如 STM32F103 系列的 **ADC 转换时间可达 1us**（信号频率较高时需要注意）。

输入电压范围通常为 **0-3.3V**（或 0-5V，具体取决于供电电压和参考电压设置），转换结果范围为 **0-4095**。（如果面试问：就说是 **12 位的范围是：0 到 2 的 12 次方减 1**）

多个输入通道，可测量外部 GPIO 口和内部信号源（如内部温度传感器和内部参考电压）。规则组和注入组两个转换单元，是 STM32 ADC 的增强功能。

模拟看门狗自动监测输入电压范围，当 AD 值高于上阈值或低于下阈值时，会申请中断，减轻软件负担。

4.触发方式与数据对齐

触发方式可以是**软件触发**或**硬件触发**。软件触发由 ADC 的库函数完成，硬件触发则利用定时器或外部引脚申请中断进行触发。

数据对齐方式有**右对齐**和**左对齐**两种。右对齐是常用方式，读出的值就是实际值；左对齐则适用于不需要太大分辨率的场合，可以只取高 8 位数据。

9.2 DAC（数模转换器）

1.定义与作用

DAC（Digital-Analog Converter）数模转换器，是将数字信号（通常是二进制数）转换为模拟信号（如电压或电流）的过程。

2.工作原理

输入数字信号：由微控制器或其他数字设备提供的二进制数，代表需要转换的模拟量的离散级别。

量化过程：根据输入的数字信号生成与之对应的模拟信号。

电路生成模拟信号：DAC 内部使用特定的电路来输出模拟信号。

滤波与平滑：通过低通滤波器进行滤波与平滑处理，使输出为一个更连续、平滑的模拟信号。

3.STM32 DAC 应用

STM32 的某些系列（如 F4 系列）具有 DAC 功能，可以**输出模拟信号，如正弦波等**。

DAC 常用于波形生成领域，如**信号发生器、音频解码芯片等**。

可以通过配置 DAC 的输出引脚、输出缓冲等参数来满足不同的应用需求。、

9.3 面试复习建议

1.ADC 相关问题

什么是 ADC，它的工作原理是什么？

ADC（Analog-to-Digital Converter）是将**模拟信号（如电压或电流）转换为数字信号**的电子设备。其工作原理通常包括**采样、保持、量化和编码**四个步骤。

STM32 的 ADC 有哪些主要特点？

STM32 的 ADC 通常具有 **12 位或更高的分辨率**，转换速度快，支持多个输入通道，可测量外部信号和内部信号源。此外，还具有自动校准、模拟看门狗等功能。

如何配置 STM32 的 ADC？

配置 STM32 的 ADC 通常涉及**选择输入通道、设置采样时间、选择触发方式（软件触发或硬件触发）、配置数据对齐方式等**。这些配置可以通过 **STM32 的库函数或寄存器**来完成。

在 STM32 项目中，你如何使用 ADC？

在 STM32 项目中，ADC 常用于读取模拟传感器的数据，如**温度传感器、压力传感器等**。通过配置 ADC，可以读取这些传感器的模拟信号，并将其转换为数字信号进行处理。

ADC 的采样率和分辨率是如何影响测量结果的？

采样率决定了 ADC 每秒可以采样的次数，而分辨率决定了 ADC 可以区分的最小模拟信号变化量。采样率越高，可以捕获的信号细节越多；分辨率越高，测量结果的精度越高。

DAC 相关问题

什么是 DAC，它的工作原理是什么？

DAC（Digital-to-Analog Converter）是将数字信号转换为模拟信号的电子设备。其工作原理通常包括输入数字信号、量化过程、电路生成模拟信号以及滤波与平滑等步骤。

STM32 的 DAC 有哪些主要特点？

STM32 的 DAC 通常具有 8 位或更高的分辨率，输出速度快，支持多种输出模式（如单极性输出、双极性输出等）。此外，还具有输出缓冲、可编程输出范围等功能。

如何配置 STM32 的 DAC？

配置 STM32 的 DAC 通常涉及选择输出通道、设置输出范围、配置输出缓冲等。这些配置可以通过 STM32 的库函数或寄存器来完成。

在 STM32 项目中，你如何使用 DAC？

在 STM32 项目中，DAC 常用于生成模拟信号，如正弦波、方波等。通过配置 DAC 的输出参数，可以生成所需的模拟信号，并用于驱动外部设备或进行信号测试。

DAC 的分辨率和输出速度是如何影响输出信号的？

DAC 的分辨率决定了输出信号的精度，分辨率越高，输出信号的精度越高。输出速度则决定了 DAC 可以生成模拟信号的速度，输出速度越快，可以生成的模拟信号频率越高。